

Interrupt / DMA × OS I/O

设备异步完成怎样变成内核可确认的 request completion 与任务唤醒

408 · Cross-Subject X-B03

Mother Interface

OS Driver Submit

DMA Mapping / Descriptor

DMA

Interrupt / Poll

Software Completion

Device Transfer

Memory Buffer + Completion State

Driver Verify

Wake / CQ / Callback

本 Bridge 专门负责连接“计算机组成原理（I/O 接口、DMA 控制器、中断硬件通路）”与“操作系统（I/O 子系统、设备驱动、缓冲区管理与等待唤醒）”的交互契约；磁盘底层物理结构与总线仲裁分别回到各自专题。

先定义接口两侧的词

- **I/O Request (I/O 请求描述符)**：操作系统为一次 I/O 交互分配的元数据结构 (如 Linux struct bio / request)，保存目标设备、物理页缓冲区、偏移量与完成回调。
- **DMA (Direct Memory Access, 直接内存访问)**：设备或 DMA 引擎在 CPU/驱动建立请求与可访问映射后，直接在设备与内存之间搬运数据；CPU 不必用通用寄存器逐字节参与，但仍负责请求建立、描述符提交、同步与完成处理。
- **DMA-visible Mapping (设备可见映射)**：驱动必须让设备获得在本次传输生命周期内有效的 DMA address / descriptor。经典 408 模型常把它直接视为主存物理地址；现代平台还可能经过 IOMMU 或总线地址翻译，因此不能把 DMA address 与 CPU physical address 永久画等号。
- **Buffer Ownership (缓冲区所有权)**：提交后到完成前，CPU 与设备对缓冲区的读写必须遵守传输方向、平台 DMA API 与一致性协议；稳定要求是双方不能以会破坏本次传输语义的方式并发修改同一数据，而不是跨所有平台绝对化为“CPU 一律不能碰缓冲区”。
- **Completion Evidence (完成证据)**：设备状态寄存器中的成功标志位或完成队列描述符；操作系统不能仅凭“中断信号到达”就断定数据有效，必须由驱动读取/消费相应完成状态核实。

Owners 与职责边界

- **左侧 Owner (CO-08 I/O 系统)**：拥有设备控制器寄存器（控制/状态/数据）、DMA 计数器与主存物理地址寄存器、中断引脚与仲裁。
- **右侧 Owner (OS-05 I/O 管理)**：拥有设备独立性软件、设备驱动程序、I/O 请求排队调度、阻塞任务休眠与唤醒（OS-B01）。
- **本 Bridge Owns**：DMA 传输启动配置、缓冲区所有权交接、中断完成确认与任务状态迁移契约。

1 核心机制：缓冲区所有权 (Buffer Ownership) 交接契约

在中断驱动与 DMA 模式下，CPU 与外部设备呈现高度异步解耦：

1. 所有权转移 (OS → DMA 控制器):

- OS 设备驱动准备缓冲区并建立本次传输所需的 DMA-visible mapping / descriptor; 在经典 408 直连模型中可把 DMA address 视为目标主存地址, 现代平台则可能经 IOMMU; 同时写入方向、长度、描述符/队列位置等控制信息;
- 提交请求后, 设备获得按协议访问 Buffer 的资格。调用线程是否阻塞是 API/请求语义的另一条轴: 阻塞式 I/O 可能 sleep 让出 CPU, 异步/非阻塞提交也可以先返回, 不能由“使用 DMA”单独推出 Blocked。

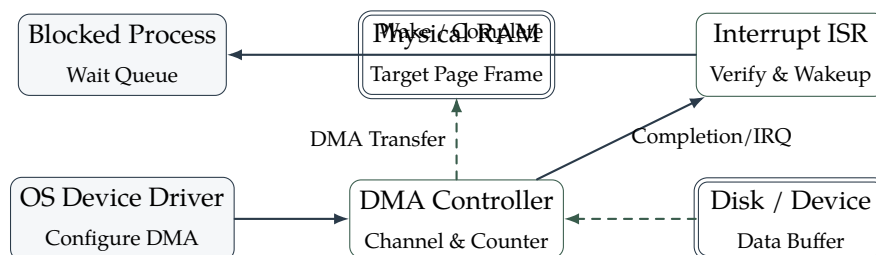
2. 硬件直接搬运 (卸载逐字节 CPU 搬运):

- 设备/DMA 引擎按描述符直接与内存交换数据; 经典总线模型可表现为周期挪用 (Cycle Stealing)、突发/块传送等方式;
- CPU 不用执行逐字节 load/store 搬运循环, 可以并发执行其他工作, 但 DMA 仍会消耗内存/互连带宽, 并可能带来 cache/IOMMU 同步成本, 因此不等于“零 CPU/零系统成本”。

3. 所有权回收与完成确认 (DMA 控制器 → OS):

- 设备把完成结果写入状态寄存器/完成队列, 并可能通过中断通知 CPU; 有些高性能路径也会由软件 polling completion queue, 因此 DMA completion 与 interrupt 是两种不同事件;
- 若采用中断, CPU 进入驱动 ISR/完成处理; 驱动读取并消费设备完成证据、处理错误与同步要求;
- 对阻塞式 waiter, 完成路径可调用 wakeup() 使其重新进入 Ready/Runnable; 对异步请求, 则可更新 completion queue/callback 等其他软件状态。

2 母模型: DMA 控制器与操作系统 I/O 交互拓扑



3 四种 I/O 控制方式全景对比

控制方式	数据传输单位	CPU 参与度与开销	适用场景与硬件成本
程序轮询 (Polling)	字 / 字节	CPU 全程死等状态寄存器，极其空耗算力	极低速设备 (早年打印机) 或超高速设备无切换
中断驱动 (Interrupt)	字 / 字节	每次传输一个字触发一次中断，频繁切换上下文	中低速字符设备 (键盘、鼠标、串口)
DMA 控制 (DMA)	连续物理数据块 (Block)	仅在块传输启动与结束时介入两次，中间零参与	高速块设备 (磁盘、SSD、网卡)；成本适中
通道控制 (Channel)	一组复杂数据块列表	CPU 发送通道指令后完全脱身，通道独立执行通道程序	大型主机与复杂存储阵列；硬件成本高

4 阻塞磁盘读取的完整时序轨迹

1. **用户调用发起**：应用进程调用 `read(fd, buf, 4096)` 陷入内核；
2. **驱动准备与启动**：文件系统定位磁盘块号 (LBN)，驱动将物理页框地址写入 DMA 目标地址寄存器，写入块数 1，向控制寄存器写入启动读命令；
3. **进程休眠让出**：内核将当前线程移入该设备的等待队列，状态置为 `TASK_UNINTERRUPTIBLE`，调度器切换至其他就绪线程运行；
4. **硬件后台搬运**：磁盘寻道旋转，控制器将扇区数据经总线直接写入物理内存，计数器减至 0；
5. **中断响应与验核**：DMA 向中断控制器发送中断信号；CPU 暂停当前指令流执行 ISR；驱动读取状态字确认无校验错，标记 `request` 完成；
6. **唤醒与返回**：调用 `wakeup()` 将休眠线程移入 Run Queue；调度器选中该线程后，数据已在内存中就绪，系统调用正常返回用户态。

Anti-Bridge：三个必须阻断的误区

- DMA ≠ CPU 内存拷贝：DMA 把逐字节/逐字搬运从 CPU 指令流卸载给设备/DMA 引擎，但 CPU 仍负责 `setup`、`submission`、`completion` 与资源回收。
- 中断到达 ≠ 数据已保证正确：中断只说明有设备事件需要处理；驱动仍须消费完成队列/状态并确认本次 `request` 的结果。反过来，DMA `completion` 也不要求所有系统都用“一次完成一次中断”的发现方式。
- DMA 地址 ≠ 用户虚拟地址：用户 VA 不能直接作为设备访问契约；驱动必须建立设备可用的 DMA-visible mapping。经典教材可按“物理主存地址”推演，现代平台则还可能经过 IOMMU，因此也不要再推出 `DMA address = CPU PA` 的普遍结论。

30 秒极速调用协议

做题遇到“I/O 控制方式、DMA 传输周期与中断处理”时，严格按三步判定：

Submit DMA-visible mapping / descriptor

设备搬运并产生完成证据

Interrupt/Poll 发现并由驱动核验、交付软件完